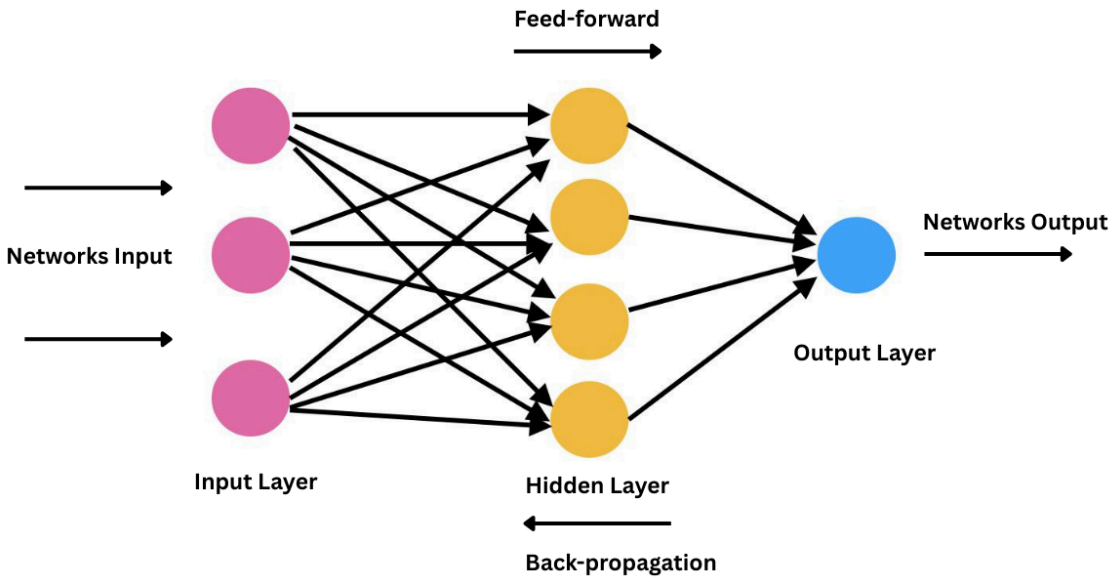
Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Dipersiapkan Oleh Tim Asisten IF3270 2025/2026

Versi: 1.0 01/03/2026



Deadline: Senin, 16 Maret 2026 23.59 WIB

Tujuan

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan *Feedforward Neural Network* (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan *FFNN from scratch*.

Spesifikasi

Implementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

- FFNN yang diimplementasikan dapat **menerima jumlah neuron dari tiap layer** (termasuk input layer dan output layer).
- FFNN yang diimplementasikan dapat **menerima fungsi aktivasi dari tiap layer**. Pilihan fungsi aktivasi yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
Linear	$Linear(x) = x$
ReLU	$ReLU(x) = \max(0, x)$
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
Hyperbolic Tangent (tanh)	$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

- FFNN yang diimplementasikan dapat **menerima fungsi loss** dari model tersebut. Pilihan loss function yang harus diimplementasikan adalah sebagai berikut:

Nama Fungsi Aktivasi	Definisi Fungsi
MSE	$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
Binary Cross-Entropy	$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$ y_i = Actual binary label (0 or 1) $\hat{y}_i$ = Predicted value of y_i n = Batch size
Categorical Cross-Entropy	$$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$$ y_{ij} = Actual value of instance i for class j $\hat{y}_{ij}$ = Predicted value of y_{ij} C = Number of classes n = Batch size

  - Catatan:
    - Binary cross-entropy merupakan kasus khusus categorical cross-entropy dengan kelas sebanyak 2.
    - Log yang digunakan merupakan logaritma natural (logaritma dengan basis e).
- Terdapat mekanisme untuk **inisialisasi bobot** tiap neuron (termasuk bias). Pilihan metode inisialisasi bobot yang harus diimplementasikan adalah sebagai berikut:
  - **Zero initialization**
  - Random dengan distribusi **uniform**.
    - Menerima parameter lower bound (batas minimal) dan upper bound (batas maksimal).

        - Menerima parameter seed untuk reproducibility.
    - Random dengan distribusi **normal**.
        - Menerima parameter mean dan variance.
        - Menerima parameter seed untuk reproducibility.
- Instance model yang diinisialisasikan harus bisa **menyimpan bobot** tiap neuron (termasuk bias).
- Instance model yang diinisialisasikan harus bisa **menyimpan gradien bobot** tiap neuron (termasuk bias).
- Instance model memiliki method untuk **menampilkan distribusi bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk **menampilkan distribusi gradien bobot** dari tiap layer.
    - Menerima masukan berupa list of integer (bisa disesuaikan ke struktur data lain sesuai kebutuhan) yang menyatakan layer mana saja yang distribusinya akan di-plot.
- Instance model memiliki method untuk ***save*** dan ***load***.
- Model memiliki implementasi **forward propagation** dengan ketentuan sebagai berikut:
    - Dapat menerima input berupa **batch**.
- Model memiliki implementasi **backward propagation** untuk menghitung perubahan gradien:
    - Dapat menangani perhitungan perubahan gradien untuk input data **batch**.
    - Gunakan konsep **chain rule** untuk menghitung gradien tiap bobot terhadap loss function.
    - Berikut merupakan **turunan pertama** untuk setiap fungsi aktivasi:

Nama Fungsi Aktivasi	Turunan Pertama
Linear	$\frac{d\big(Linear(x)\big)}{dx} = 1$
ReLU	$\frac{d\big(ReLU(x)\big)}{dx} = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$
Sigmoid	$\frac{d\big(\sigma(x)\big)}{dx} = \sigma(x)\big(1 - \sigma(x)\big)$

Hyperbolic Tangent (tanh)	$$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^x - e^{-x}}\right)^2$$
Softmax	Untuk vector $\vec{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $$\frac{d(softmax(\vec{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\vec{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$$ Dimana untuk $i, j \in \{1, \dots, n\}$, $$\frac{\partial(softmax(\vec{x})_i)}{\partial x_j} = softmax(\vec{x})_i\left(\delta_{i,j} - softmax(\vec{x})_j\right)$$ $$\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$

- Berikut merupakan **turunan pertama** untuk setiap fungsi loss terhadap bobot suatu FFNN (lanjutkan sisanya menggunakan chain rule):

Nama Fungsi Loss	Definisi Fungsi
MSE	$$\frac{\partial \mathcal{L}_{MSE}}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)\frac{\partial \hat{y}_i}{\partial W}$$
Binary Cross-Entropy	$$\frac{\partial \mathcal{L}_{BCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\partial \mathcal{L}_{BCE}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}\frac{\partial \hat{y}_i}{\partial W}$$
Categorical Cross-Entropy	$$\frac{\partial \mathcal{L}_{CCE}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{\partial \mathcal{L}_{CCE}}{\partial \hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}\frac{y_{ij}}{\hat{y}_{ij}}\frac{\partial \hat{y}_{ij}}{\partial W}$$

- Model memiliki implementasi metode **regularisasi L1** dan **L2**.
- Model memiliki implementasi **weight update** dengan menggunakan **gradient descent** untuk memperbarui bobot berdasarkan gradien yang telah dihitung, berikut persamaannya:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial \mathcal{L}}{\partial W_{old}} \right)$$

$$\alpha = \text{Learning rate}$$

- Implementasi untuk pelatihan model harus memenuhi ketentuan berikut:
    - Dapat menerima parameter berikut:
        - Batch Size
        - Learning Rate
        - Jumlah Epoch
        - Verbose
- Verbose 0 berarti tidak menampilkan apa-apa selama pelatihan.
- Verbose 1 berarti hanya menampilkan progress bar beserta dengan kondisi training loss dan validation loss saat itu.
    - Proses pelatihan mengembalikan **histori dari proses pelatihan** yang berisi **training loss** dan **validation loss tiap epoch**

Pengujian

Gunakan dataset berikut untuk menguji model: global_student_placement_and_salary.

Pengujian dilakukan di **file .ipynb terpisah**.

1. **Hyperparameter**

   Analisis pengaruh beberapa **hyperparameter** sebagai berikut:

   - Pengaruh **depth (banyak layer)** dan **width (banyak neuron per layer)**
     - Pilih 3 variasi kombinasi width (depth tetap) dan 3 variasi depth (width semua layer tetap).
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
   - Pengaruh **fungsi aktivasi** hidden layer.
     - Tentukan base arsitektur (minimal 3 layer), pilih 1 layer sebagai layer pengetesan.
     - Lakukan eksperimen dengan menjalankan variasi base arsitektur dengan setiap fungsi aktivasi yang diimplementasikan **kecuali softmax** pada layer pilihan.
     - Bandingkan hasil akhir prediksinya.
     - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.

        - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
    - Pengaruh **learning rate**.
        - Lakukan 3 variasi learning rate (nilainya dibebaskan).
        - Bandingkan hasil akhir prediksinya.
        - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
        - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.

2. **Pengaruh Regularisasi**.
    - Lakukan eksperimen masing-masing 1 kali untuk model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.
    - Bandingkan hasil akhir prediksinya.
    - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
    - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
3. **Uji Perbandingan**.

    Analisis perbandingan hasil prediksi dengan library sklearn MLP.

        - Lakukan satu kali pelatihan dengan hyperparameter yang sama untuk kedua model.
        - Hyperparameter yang digunakan dibebaskan.
        - Bandingkan hasil akhir prediksinya saja.

Informasi Dataset

Pada Tugas Besar ini, Anda juga diminta untuk melakukan **analisis dan pemodelan** menggunakan dataset *Global Student Placement & Salary Dataset* menggunakan **FFNN** yang sudah kalian buat. Dataset ini berisi kumpulan data global mengenai performa akademik mahasiswa serta outcome penempatan kerja dan gaji setelah lulus. Dataset dapat diakses **di sini**. Berikut deskripsi dari tiap fitur yang ada pada dataset yang akan digunakan (**placement_status** adalah kolom yang perlu diprediksi):

Column	Description	Type
cgpa	Cumulative Grade Point Average (skala 4.0–10.0)	Float
backlogs	Jumlah mata kuliah yang belum lulus	Integer
college_tier	Tingkatan institusi	Categorical

country	Negara kelulusan	Categorical
university_ranking_band	Kategori peringkat global universitas	Categorical
internship_count	Jumlah internship yang telah diselesaikan	Integer
aptitude_score	Skor tes kemampuan kuantitatif	Integer
communication_score	Skor kemampuan komunikasi	Integer
specialization	Bidang akademik	Categorical
industry	Industri tempat bekerja	Categorical
internship_quality_score	Skor kualitas pengalaman internship	Float
placement_status	Status penempatan kerja	Categorical

Spesifikasi Bonus

Berikut merupakan beberapa spesifikasi bonus yang dapat Anda kerjakan:

- **(40% total bonus)** Implementasikan FFNN dengan menggunakan **automatic differentiation**.
    - Metode ini merupakan metode yang umum digunakan untuk perhitungan gradien pada library-library untuk deep learning seperti PyTorch atau TensorFlow.
    - Jika ingin mengimplementasikan bonus ini, sangat disarankan untuk mengimplementasikan dari sejak awal mengerjakan tugas supaya tidak terlalu banyak perubahan dari implementasi biasa.
    - Referensi video:
      The spelled-out intro to neural networks and backpropagation: building mic…
- **(5% total bonus)** Implementasikan minimal 2 fungsi aktivasi lain yang sering digunakan.
- **(5% total bonus)** Implementasikan 2 metode inisialisasi bobot berikut:
    - Xavier initialization.
    - He initialization.
- **(10% total bonus)** Implementasikan metode normalisasi RMSNorm, kemudian lakukan analisis untuk beberapa hal berikut:
    - Lakukan eksperimen masing-masing 1 kali untuk model tanpa normalisasi dan dengan normalisasi.
    - Bandingkan hasil akhir prediksinya.
    - Bandingkan grafik training loss dan validation loss tiap epoch setelah pelatihan.
    - Bandingkan distribusi bobot dan gradien bobot dari beberapa/semua layer pada model.
- **(40% total bonus)** Implementasikan optimizer Adam (Adaptive Moment Estimation) sebagai alternatif dari Gradient Descent standar dan lakukan **analisis perbandingan**

kecepatan konvergensi antara Gradient Descent standar dengan Adam pada dataset yang diberikan. Link paper: [link]

Kelompok

Pembagian kelompok ditentukan sendiri oleh mahasiswa dengan mengisi sheets kelompok IF3270 Pembelajaran Mesin berikut ini dengan 1 kelompok terdiri dari **3 orang** dan **tidak lintas kelas.** Batas waktu pengisian kelompok adalah **Rabu, 4 Maret 2026 pukul 23:59 WIB**. Setelah waktu yang ditentukan, mahasiswa yang belum mengisi sheets kelompok akan diacak.

QnA

Pertanyaan dapat ditanyakan pada **Teams**. Pastikan pertanyaan yang ditanyakan tidak berulang.

Aturan

Terdapat beberapa hal yang harus diperhatikan dalam pengerjaan tugas ini, yakni:

1. Jika terdapat hal yang tidak dimengerti, silahkan ajukan pertanyaan kepada asisten melalui **reply post Tugas Besar 1 di Teams IF3270**.
2. Dilarang melakukan **plagiarisme, menggunakan AI dalam bentuk yang dilarang dalam aturan ITB, dan melakukan kerjasama antar kelompok**. Pelanggaran pada poin ini akan menyebabkan pemberian **nilai E** pada setiap anggota kelompok.
3. Tugas diimplementasikan dalam bahasa **Python**.
4. Implementasi FFNN *from scratch* hanya boleh menggunakan library untuk perhitungan matematika (Contoh: NumPy, dll).

Deliverables

- Tugas dikumpulkan dalam bentuk link ke *repository* GitHub yang **minimal** berisi beberapa hal berikut (boleh ditambahkan jika dirasa perlu):
    - Folder **src**, digunakan untuk menyimpan source code implementasi FFNN beserta dengan notebook pengujian.
    - Folder **doc,** digunakan untuk menyimpan laporan dalam bentuk **.pdf** yang terdiri atas komponen berikut:
        - Cover
        - Deskripsi Persoalan
        - Pembahasan
            - Penjelasan Implementasi
                - Deskripsi kelas beserta deskripsi atribut dan methodnya.

            - Penjelasan forward propagation.
            - Penjelasan backward propagation dan weight update.
        - Hasil Pengujian
            - Pengaruh depth dan width.
            - Pengaruh fungsi aktivasi.
            - Pengaruh learning rate.
            - Pengaruh inisialisasi bobot.
            - Pengaruh regularisasi.
            - Pengaruh normalisasi RMSNorm (jika mengerjakan).
            - Perbandingan dengan library sklearn.
    - Kesimpulan dan Saran
    - Pembagian tugas tiap anggota kelompok
    - Referensi
    - Lampiran Form Penggunaan AI (Hal. 19 pada panduan AI ITB)
  - **README.md**, yang berisi deskripsi singkat repository, cara setup dan run program, dan pembagian tugas tiap anggota kelompok.
- Pengumpulan dilakukan melalui **edunex.**
- Jika ada masalah pada **edunex,** maka gunakan form pengumpulan berikut: Form Pengumpulan Tugas Besar 1 IF3270 Pembelajaran Mesin
- Batas akhir pengumpulan adalah hari **Senin, 16 Maret 2026 23.59 WIB.** Tugas yang terlambat dikumpulkan tidak akan diterima.
- Pengumpulan dilakukan oleh **NIM terkecil**.

Referensi

- The spelled-out intro to neural networks and backpropagation: building micrograd
- https://www.jasonosajima.com/forwardprop
- https://www.jasonosajima.com/backprop
- https://numpy.org/doc/2.2/
- https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.Html
- https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions
- https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/
- https://douglasorr.github.io/2021-11-autodiff/article.html
- https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
- https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/
- https://builtin.com/data-science/l2-regularization